1 Inertial aiding

With an IMU configured, a first-order strapdown mechanization in the Earth-centred Earth-fixed (ECEF) frame replaces the inter-epoch time update of the time update. Two switches control it. `imu.enabled` enables the IMU. `estimation.aiding.imu` selects the IMU source in the `AidingSourceManager`. When `estimation.aiding.imu` is absent it defaults to the value of `imu.enabled`; an explicit value takes precedence (`src/core/config/GnssConfigLoad.cpp`).

The mechanization is implemented in `src/core/filter/ImuIntegrator.cpp`. The per-epoch prediction runs `Application::predictEpoch_` (`src/core/app/PppEpochSteps.cpp`) → `AidingSourceManager::collectPredict` → `src/core/aiding/ImuAidingSource.cpp`, and `KalmanFilter::predictAided` (`src/core/filter/KalmanFilter.cpp`) splices the resulting `PredictContribution` into the filter covariance (`KalmanFilter::spliceAndPropagate` in `src/core/filter/KalmanKernels.cpp`). The real-time engine (`RealTimeEngine::processEpoch`) uses the same `predictAided` / `collectPredict` path. Only the sample ingest differs. Post-processing reads a CSV file with `loadData`, which sorts the samples by time. In real time each sensor sample reaches `ImuIntegrator::addSample` through `ImuAidingSource::ingestSample`, which appends it to the same buffer in arrival order. For samples that arrive in time order the two paths therefore give the same `integrate` results.

The `IAidingSource` / `AidingSourceManager` abstraction (`include/aiding/`) is the interface through which every non-GNSS modality contributes up to three items: a `StateManager`-owned state block, a predict-time Φ/Q sub-block, and either independent measurement rows or extra Jacobian *columns* on existing GNSS rows. The IMU is its first client; a zero-velocity update source is the second, and the registry declares no implementation for the non-holonomic, wheel, barometric and vision kinds. `ImuAidingSource` reproduces the arithmetic of the older monolithic `KalmanFilter::predictImu`, so the strapdown equations below apply unchanged. `predictImu` has been removed from the source; `predictAided` replaces it.

The formulation is an *error-state* extended Kalman filter. The mechanization propagates the nominal navigation state $(\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{C}_b^e)$ deterministically from the inertial samples, while a 15-element INS error vector

$$\delta\boldsymbol{x}_{\text{ins}} = \begin{bmatrix}\delta\boldsymbol{r} & \delta\boldsymbol{v} & \delta\boldsymbol{\psi} & \delta\boldsymbol{b}_a & \delta\boldsymbol{b}_g\end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^{15} \tag{1}$$

carries the linearised transition and process noise that propagate the covariance. Three attitude-error states $\delta\boldsymbol{\psi}$, three accelerometer-bias states $\boldsymbol{b}_a$ and three gyro-bias states $\boldsymbol{b}_g$ augment the position state (and the three velocity states when `estimation.estimate_velocity` is set). `StateManager` allocates the nine added states as one contiguous block (accessors `getAttitudeIndex`, `getAccelBiasIndex`, `getGyroBiasIndex`).

Remark 1 (Implemented scope). `ImuIntegrator` fixes the block order of Equation (1) ($[\delta\boldsymbol{r}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$). The filter carries no lever-arm state and no sensor scale-factor states, and the GNSS measurement model has no IMU-to-antenna lever-arm term (Section 1.4). These are planned extensions. The equations below omit the corresponding terms.

1.1 Strapdown mechanization

For each IMU sample, the raw specific force $\tilde{\boldsymbol{a}}$ and angular rate $\tilde{\boldsymbol{\omega}}$ (both in the body frame) are corrected with the current filter bias estimates,

$$\boldsymbol{f}_b = \tilde{\boldsymbol{a}} - \boldsymbol{b}_a, \qquad \boldsymbol{\omega}_b = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g, \tag{2}$$

and integrated over the sample interval Δt. A sample whose interval is not positive or exceeds 10 s is skipped. The attitude direction-cosine matrix $\boldsymbol{C}_b^e$ (body $\rightarrow$ ECEF) is updated to first order with Earth-rotation compensation,

$$\boxed{\boldsymbol{C}_b^e \;\leftarrow\; \boldsymbol{C}_b^e\big(\boldsymbol{I} + [\boldsymbol{\omega}_b]_\times\,\Delta t\big) \;-\; [\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{C}_b^e\,\Delta t,} \tag{3}$$

where $\boldsymbol{\omega}_{ie}^e = [\,0,\,0,\,\omega_e\,]^{\mathrm{T}}$ is the ECEF Earth-rotation vector ($\omega_e = 7.292\,115\,146\,7 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$, `common/Constants.hpp`) and $[\cdot]_\times$ is the skew-symmetric (cross-product) matrix. To limit numerical drift, $\boldsymbol{C}_b^e$ is re-orthonormalised by singular-value decomposition every 100 integrated samples, starting with the first. The velocity and position propagate as

$$\boldsymbol{v}_{k+1} = \boldsymbol{v}_k + \big(\boldsymbol{C}_b^e\,\boldsymbol{f}_b + \boldsymbol{g}^e(\boldsymbol{r}_k) - 2\,[\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{v}_k\big)\Delta t, \tag{4}$$

$$\boldsymbol{r}_{k+1} = \boldsymbol{r}_k + \tfrac{1}{2}\big(\boldsymbol{v}_k + \boldsymbol{v}_{k+1}\big)\Delta t. \tag{5}$$

The specific force is resolved with the attitude already updated by Equation (3), and gravity is evaluated at the position of the start of the step. Equation (4) applies the Coriolis acceleration $-2\,[\boldsymbol{\omega}_{ie}^e]_\times\boldsymbol{v}$. The centrifugal acceleration is part of the gravity vector $\boldsymbol{g}^e$: $\boldsymbol{g}^e$ is a *plumb-bob* gravity (gravitational plus centrifugal). Equation (5) is a trapezoidal position update.

The gravity vector is a point-mass radial model plus the centrifugal term (`ImuIntegrator::computeGravityEcef`):

$$\boldsymbol{g}^e(\boldsymbol{r}) = -\,\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r} \;+\; \omega_e^2\,[\,p_x,\,p_y,\,0\,]^{\mathrm{T}}, \qquad \mu = 3.986\,005 \times 10^{14}\,\mathrm{m^3\,s^{-2}}. \tag{6}$$

The normal-gravity (Somigliana) flattening and the J_2 zonal term are not modelled. Over one GNSS epoch interval the resulting deterministic acceleration error is small, and the estimated accelerometer bias $\boldsymbol{b}_a$ absorbs it.

1.2 INS error-state dynamics

Linearising Equations (3) to (5) about the nominal trajectory yields the continuous error dynamics $\dot{\delta\boldsymbol{x}}_{\mathrm{ins}} = \boldsymbol{F}_{\mathrm{ins}}\,\delta\boldsymbol{x}_{\mathrm{ins}} + \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{w}$, with, in the block order of Equation (1),

$$\boldsymbol{F}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -2[\boldsymbol{\omega}_{ie}^e]_\times & -[\boldsymbol{f}^e]_\times & \boldsymbol{C}_b^e & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & -[\boldsymbol{\omega}_{ie}^e]_\times & \boldsymbol{0} & -\boldsymbol{C}_b^e \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{f}^e := \boldsymbol{C}_b^e\,\boldsymbol{f}_b. \tag{7}$$

Both sensor biases are random walks (no deterministic dynamics: the null bottom rows of Equation (7)). Two terms of the standard ECEF error model are omitted by design: the position-error feedback row ($[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{r},\delta\boldsymbol{r}} = -[\boldsymbol{\omega}_{ie}^e]_\times$) and the gravity-gradient coupling $[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{v},\delta\boldsymbol{r}} = +\,\partial\boldsymbol{g}^e/\partial\boldsymbol{r}$ (the gradient of the *plumb* gravity of Equation (6), which already subsumes the centrifugal $-[\boldsymbol{\omega}_{ie}^e]_\times^2$; the leading sign follows from differentiating the velocity update Equation (4)). The resulting first-order model is accurate over the short epoch interval of a GNSS-aided run. The discrete transition is also first order,

$$[\boldsymbol{\Phi}_{\mathrm{ins}}]_k = \boldsymbol{I} + \boldsymbol{F}_{\mathrm{ins}}\,\Delta t. \tag{8}$$

The process noise maps the continuous accelerometer and gyro white noise and the bias random walks through

$$[\boldsymbol{Q}_{\text{ins}}]_k = \boldsymbol{G}_{\text{ins}}\,\boldsymbol{W}\,\boldsymbol{G}_{\text{ins}}^{\mathrm{T}}\,\Delta t, \quad \boldsymbol{G}_{\text{ins}} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \boldsymbol{C}_b^e & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\boldsymbol{C}_b^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \boldsymbol{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \boldsymbol{I} \end{bmatrix}, \quad \boldsymbol{W} = \operatorname{diag}\!\big(\sigma_a^2, \sigma_g^2, \sigma_{ba}^2, \sigma_{bg}^2\big) \otimes \boldsymbol{I}_3, \tag{9}$$

where the spectral densities $\sigma_a, \sigma_g, \sigma_{ba}, \sigma_{bg}$ are read from `imu.{accel_noise_density, gyro_noise_density, accel_bias_instability, gyro_bias_instability}`. The defaults in `include/config/GnssConfigTypes.hpp` are $0.05\,\mathrm{m\,s^{-2}\,Hz^{-0.5}}$, $5 \times 10^{-4}\,\mathrm{rad\,s^{-1}\,Hz^{-0.5}}$, $1 \times 10^{-4}\,\mathrm{m\,s^{-3}\,Hz^{-0.5}}$ and $1 \times 10^{-6}\,\mathrm{rad\,s^{-2}\,Hz^{-0.5}}$, in that order. Over one GNSS epoch the per-sample pairs $([\boldsymbol{\Phi}_{\text{ins}}]_k, [\boldsymbol{Q}_{\text{ins}}]_k)$ accumulate by the standard recursion

$$\boldsymbol{\Phi}_{\text{ins}} \leftarrow [\boldsymbol{\Phi}_{\text{ins}}]_k\,\boldsymbol{\Phi}_{\text{ins}}, \qquad \boldsymbol{Q}_{\text{ins}} \leftarrow [\boldsymbol{\Phi}_{\text{ins}}]_k\,\boldsymbol{Q}_{\text{ins}}\,[\boldsymbol{\Phi}_{\text{ins}}]_k^{\mathrm{T}} + [\boldsymbol{Q}_{\text{ins}}]_k. \tag{10}$$

1.3 Covariance splice and attitude feedback

The accumulated 15×15 pair $(\boldsymbol{\Phi}_{\text{ins}}, \boldsymbol{Q}_{\text{ins}})$ is written over the indices the INS owns in an identity-seeded full-state transition $\boldsymbol{F}$ and in the random-walk process-noise baseline $\boldsymbol{Q}$. The write is an assignment, so the baseline noise on those rows is not counted twice. The baseline process noise is evaluated over the interval the INS integrated. The non-INS states (clock, troposphere, ionosphere, ambiguities, ...) keep their random-walk model from the time update; because the transition is the identity there, their means are not propagated. The covariance is then propagated once and symmetrised,

$$\boxed{\boldsymbol{P} \leftarrow \boldsymbol{F}\,\boldsymbol{P}\,\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{Q}, \qquad \boldsymbol{P} \leftarrow \tfrac{1}{2}\big(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}}\big).} \tag{11}$$

The mechanised values of Equations (4) to (5) overwrite the nominal navigation states $(\boldsymbol{r}, \boldsymbol{v})$. The attitude error is reset, $\delta\boldsymbol{\psi} := \mathbf{0}$, because Equation (3) has already applied the rotation increment to $\boldsymbol{C}_b^e$: after each aided prediction the IMU aiding source writes the propagated attitude into the filter through its attitude setter. The filter holds the single nominal attitude. It is initialised to the identity (there is no initial alignment step), and on the live path only two operations write it: that setter call and the feedback step below. After the next GNSS measurement update, the corrected attitude error feeds back multiplicatively into the DCM and is reset again,

$$\boldsymbol{C}_b^e \leftarrow \big(\boldsymbol{I} + [\delta\boldsymbol{\psi}]_\times\big)\boldsymbol{C}_b^e, \qquad \delta\boldsymbol{\psi} := \mathbf{0}, \tag{12}$$

followed by SVD re-orthonormalisation (`KalmanFilter::update`). This closed-loop reset keeps $\delta\boldsymbol{\psi}$ a small angle, so the linearisation in Equation (7) remains valid.

1.4 Coupling modes and the GNSS update

The coupling mode (`imu.coupling_mode`) determines how the GNSS observations correct the inertial errors. In both modes the filter processes the raw GNSS observations; the modes differ in the Doppler Jacobian and in the relinearisation path (see the remark at the end of this section).

Loosely coupled (default). Among the INS states, the GNSS Jacobians of the observation geometry touch only the position and velocity columns. The INS error states $\delta\boldsymbol{\psi}, \boldsymbol{b}_a, \boldsymbol{b}_g$ have no direct observation partials. They are corrected indirectly, through the cross-covariances that Equation (11) builds between the position/velocity block and the attitude/bias block. The feedback of Equation (12) then applies the attitude correction.

Tightly coupled. Tight coupling requires the velocity block, so the loader enables it regardless of `estimation.estimate_velocity`. A `TightCoupled` `coupling_mode` with the flag false sets the flag true and logs `"forcing estimate_velocity=true"` (`GnssConfigLoad.cpp`). The three velocity states, and with them the Doppler rows that carry the Jacobian below, are therefore always present. In this mode the Doppler observation, and only the Doppler observation, gains direct attitude and accelerometer-bias columns. It is the one aiding contribution that adds columns to existing GNSS rows instead of appending rows of its own. Each relinearisation pass calls `AidingSourceManager::applyTcJacobianInputs` → `ImuAidingSource::applyTcJacobianInputs`, which passes the current attitude and specific force to `MeasurementModel::setImuState` (`src/core/measurement/MeasurementModel.cpp`), and `MeasurementModel::buildDopplerRow` (`src/core/measurement/MeasurementModelRow.cpp`) writes the columns. With $\boldsymbol{f}^e = \boldsymbol{C}_b^e \boldsymbol{f}_b$ the specific force in ECEF and $\boldsymbol{e}_r^i$ the unit line-of-sight of the line-of-sight definition, the Doppler measurement function h_{D} contributes

$$\boxed{\frac{\partial h_{\mathrm{D}}}{\partial \delta \boldsymbol{\psi}} = -\left[\boldsymbol{f}^e\right]_{\times}^{\mathrm{T}} \boldsymbol{e}_r^i, \qquad \frac{\partial h_{\mathrm{D}}}{\partial \boldsymbol{b}_a} = -\left(\boldsymbol{C}_b^e\right)^{\mathrm{T}} \boldsymbol{e}_r^i.} \tag{13}$$

The specific force used in Equation (13) is the static-rest approximation $\boldsymbol{f}_b \approx (\boldsymbol{C}_b^e)^{\mathrm{T}}(-\boldsymbol{g}^e) - \boldsymbol{b}_a$ (`ImuAidingSource.cpp`), evaluated at the current position estimate rather than from the IMU samples. It is exact only when the true platform acceleration is negligible compared with gravity. In either mode the code and carrier-phase rows carry no attitude or bias partials; only the Doppler rows couple directly to the inertial errors.

Remark 2 (Relinearisation under tight coupling). The tightly coupled path keeps a single *naive* second-pass relinearisation instead of the prior-anchored iterated EKF of the iterated EKF. Re-deriving the gain from a fixed prior is inconsistent with the multiplicative attitude feedback of Equation (12), which has already changed $\boldsymbol{C}_b^e$ and zeroed the attitude error between passes. The aiding layer exposes the requirement as a capability query (`AidingSourceManager::anySourceRequiresNaiveRelinearization`, true for the IMU source under tight coupling), so any later source with nominal-state feedback selects the same path. Direct `tightlyCoupledImu()` checks remain in `src/core/app/PppEpochUpdate.cpp` and `src/core/filter/KalmanFilter.cpp` (the IEKF gating and the prior capture), so the migration to the capability query is incomplete.